Table of Contents

Chapter – 3: Web Mining 2

3.1 Introduction to Web-Mining 2

3.1.1 Inverted Indices and Compression 2

3.1.2 Latent Semantic Indexing 11

3.1.3 Web Search 17

3.2 Meta Search 19

3.2.1 Using Similarity Scores 20

3.2.2 Rank Positions 21

3.3 Web Spamming 24

3.3.1 Content Spamming 24

3.3.2 Link Spamming 25

3.3.3 Hiding Techniques 26

3.3.4 Combating Spam 27

Chapter – 4: Web Usage Mining 29

4.1 Web Usage Mining 29

4.2 Data Collection and Pre-Processing 30

4.3 Sources and Types of Data 31

4.4 Data Modelling 34

4.5 Discovery and Analysis of Web Usage Patterns 39

4.5.1 Session and Visitor Analysis 39

4.5.2 Cluster Analysis and Visitor Segmentation 39

4.5.3 Association and Correlation Analysis 43

4.5.4 Analysis of Sequential and Navigational Patterns 44

4.5.5 Classification and Prediction based on Web User Transactions 46

Chapter – 3: Web Mining

3.1 Introduction to Web-Mining

- Web mining is the process of discovering useful information and knowledge from web data, including hyperlinks, page contents, and usage logs.
- Given the semi-structured and unstructured nature of web data, traditional data mining techniques are often adapted or extended to address these challenges.
- The field has developed specialized algorithms and methodologies tailored to the unique characteristics of web data.

Types of Web Mining

- Web mining tasks are categorized into three main types based on the primary data used:

  - **Web Structure Mining:**
    Focuses on discovering knowledge from hyperlinks, which represent the structure of the web.

  - **Web Content Mining:**
    Extracts useful information or knowledge from the content of web pages.

  - **Web Usage Mining:**
    Mines user access patterns from usage logs, which record clicks made by users.

3.1.1 Inverted Indices and Compression

- Traditional web search and information retrieval (IR) aim to find documents containing the user's query terms.
- Scanning all documents sequentially is inefficient for large datasets like the Web. Instead, search engines use special data structures called indices to speed up retrieval.
- The most widely used and effective of these is the inverted index, which enables fast and efficient document retrieval and is quick to build.

3.1.1.1 Inverted Index

- In its basic form, an **inverted index** links each unique term to a list of documents that contain it.
- This allows quick retrieval—instantly locating documents with a given term.
- It's also straightforward to find documents that contain multiple query terms.
- The document ID (represented as id_j) identifies the document d_j that contains the term t_i, along with additional details about how t_i appears in d_j.
- The specific information stored can vary depending on the needs of the retrieval or ranking algorithm.

- For instance, to enable phrase or proximity searches, a typical posting for term t_i includes:

  $$<id_j, f_{ij}, [o_1, o_2, ..., o_{|fij|}]>,$$

  where id_j is the ID of the document containing t_i, f_{ij} is the number of times t_i appears in d_j, and *ok* represents the positions (offsets) of t_i in the document.
- These postings are arranged in ascending order by document ID and by offset within each posting, which also helps in compressing the inverted index efficiently.

Example:

- We have three documents of id_1, id_2, and id_3:

id_1: Web mining is useful.
1 2 3 4
id_2: Usage mining applications.
1 2 3
id_3: Web structure mining studies the Web hyperlink structure.
1 2 3 4 5 6 7 8

- The numbers below each document are the offset position of each word.
- The vocabulary is the set:

  {Web, mining, useful, applications, usage, structure, studies, hyperlink}

- Stopwords "is" and "the" have been removed, but no stemming is applied.

Applications:	id_2	Applications:	$<id_2, 1, [3]>$
Hyperlink:	id_3	Hyperlink:	$<id_3, 1, [7]>$
Mining:	id_1, id_2, id_3	Mining:	$<id_1, 1, [2]>, <id_2, 1, [2]>, <id_3, 1, [3]>$
Structure:	id_3	Structure:	$<id_3, 2, [2, 8]>$
Studies:	id_3	Studies:	$<id_3, 1, [4]>$
Usage:	id_2	Usage:	$<id_2, 1, [1]>$
Useful:	id_1	Useful:	$<id_1, 1, [4]>$
Web:	id_1, id_3	Web:	$<id_1, 1, [1]>, <id_3, 2, [1, 6]>$
(A)		(B)	

Two inverted indices: a simple version and a more complex version

- In the figure, (A) is a simple version, where each term is attached with only an inverted list of IDs of the documents that contain the term.
- Each inverted list in (B) is more complex as it contains additional information, i.e., the frequency count of the term and its positions in each document.

3.1.1.2 Search Using an Inverted Index

- Queries are processed by first retrieving the inverted lists for each query term, then analyzing these lists to identify documents that contain all or some of the terms.

- Query evaluation in an inverted index involves three main steps:

1. **Vocabulary Search**:
   - Each query term is searched in the vocabulary (stored in memory for speed), retrieving its inverted list.
   - Efficient structures like hashing, tries, or B-trees, and methods like binary search (O(log|V|)) are used.
2. **Results Merging**:
   - Inverted lists of all query terms are merged to find common documents.
   - The shortest list is typically used as the base, and binary search helps match it with longer lists.
   - Partial matches (documents with some, not all, terms) are also possible.
   - Since the entire index can't be stored in memory, frequently used parts are cached based on query logs.
3. **Rank Score Computation**:
   - Each matching document is assigned a relevance score using functions like Okapi or cosine similarity.
   - This score may include phrase or proximity data and is used to rank the results.

Example:

Applications: <id_2, 1, [3]>
Hyperlink: <id_3, 1, [7]>
Mining: <id_1, 1, [2]>, <id_2, 1, [2]>, <id_3, 1, [3]>
Structure: <id_3, 2, [2, 8]>
Studies: <id_3, 1, [4]>
Usage: <id_2, 1, [1]>
Useful: <id_1, 1, [4]>
Web: <id_1, 1, [1]>, <id_3, 2, [1, 6]>

- Using the inverted index from figure above, the search for the query "web mining" begins with step 1, where the inverted lists for both terms ("web" and "mining") are retrieved.

Mining: <id_1, 1, [2]>, <id_2, 1, [2]>, <id_3, 1, [3]>
Web: <id_1, 1, [1]>, <id_3, 2, [1, 6]>

- In step 2, the algorithm scans the two inverted lists and identifies documents containing both "web" and "mining" (***id₁*** and ***id₃***).
- In step 3, rank scores are calculated. Since the words appear next to each other and in the correct order in ***id₁***, it receives a higher relevance score than ***id₃***.
- Search engines may vary in how they factor in proximity and word order.

3.1.1.3 Index Construction

- Building an inverted index is straightforward and efficient, often using a trie data structure.
- The construction has a time complexity of **O(T)**, where **T** is the total number of terms (including duplicates) after preprocessing.
- Each document is scanned sequentially; for each term, the algorithm checks the trie.
- If the term exists, it updates the inverted list with the document ID and other info (like position).
- If not, a new node is added to the trie for that term.

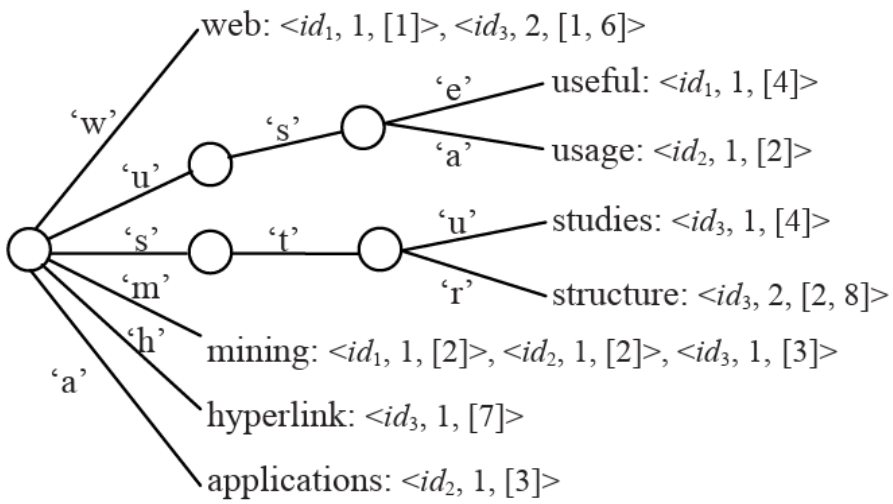
Example:

- Let us build an inverted index for the three documents below:

id_1: Web mining is useful.
1 2 3 4
id_2: Usage mining applications.
1 2 3
id_3: Web structure mining studies the Web hyperlink structure.
1 2 3 4 5 6 7 8

- To efficiently build an inverted index for large web-scale data, the entire index can't fit in memory.
- Instead, the process is done in stages:

1. **Partial Indexing**:
   - As documents are processed, a partial index (e.g., I_1) is built in memory.
   - Once memory is full, it's written to disk. This is repeated, creating multiple partial indices (I_1, I_2, ..., I_k).
2. **Hierarchical Merging**:
   - After all documents are indexed, partial indices are merged in pairs ($I_1 + I_2$, $I_3 + I_4$, etc.), then merged again at higher levels (e.g., $I_{1-2} + I_{3-4}$), continuing until a single complete index remains.
3. **Efficiency**:
   - Each merge is linear in the size of the indices due to sorted vocabularies from trie-based construction.
   - The full merging process has a time complexity of **O(k log k)**, where *k* is the number of partial indices.
4. **Optimization**:
   - To save disk space, partial indices can be merged incrementally as they are created (e.g., merge I_1 with I_2, then merge the result with I_3, and so on).



The vocabulary trie and the inverted lists

- An alternative to using a trie for indexing is to use an in-memory hash table, which is simpler and effective.
- On the Web, pages frequently change—being added, modified, or deleted—which makes updating the main index directly inefficient.
- To handle this, two **auxiliary indices** are used: one for **added pages** and one for **deleted pages**.
- Modifications are treated as a deletion followed by an addition.
- When a query is made, it's searched across:
  - $\mathbf{D_0}$: the main index,
  - $\mathbf{D_+}$: the added pages index,
  - $\mathbf{D_-}$: the deleted pages index.
- The final results are computed as:

$$(\mathbf{D_0} \cup \mathbf{D_+}) - \mathbf{D_-}$$

- When the auxiliary indices grow too large, they are merged back into the main index.

3.1.1.4 Index Compression

- Inverted indexes can be very large, so keeping them in memory is key to fast search performance.
- To reduce memory usage and avoid disk I/O, **index compression** is used.
- This technique stores the same information using fewer bits or bytes.
- With **lossless compression**, the original index can be fully reconstructed from the compressed version, significantly reducing index size without losing data.
- Inverted index compression is essential because storing integers (like document IDs and term positions) without compression uses a fixed 4-byte size, which is often unnecessary.

- Two main compression approaches are used:
    1. **Variable-bit (bitwise) schemes**:
        - encode integers using a specific number of bits (e.g., unary, Elias gamma/delta, Golomb coding).
    2. **Variable-byte schemes**:
        - store integers using full bytes, such as variable-byte coding.
- All these methods use **self-delimiting codes**, allowing the start and end of each integer to be identified without extra markers.
- An additional optimization involves **gap encoding**, where instead of storing full document IDs (which are sorted), only the difference between successive IDs is stored.
- These **gaps** are smaller and can be compressed more effectively.
- This is especially efficient for frequent terms, which tend to have small gaps.
- Gap encoding also applies to term positions (offsets), and helps significantly reduce index size.
- Each compression scheme also includes methods for both **encoding and decoding** the data.

A. Unary Coding

- Unary coding is a simple method where a number x is represented by x–1 zeros followed by a one (e.g., 5 → 00001).
- The final 1 acts as a delimiter.
- While easy to encode and decode, it's efficient only for very small numbers and inefficient for larger ones, so it's rarely used on its own in practice.
- **Example**:

    1 → 0
    2 → 10
    3 → 110
    4 → 1110

- It's very simple, but **inefficient for large numbers**, since it grows linearly.

B. Elias Gamma Coding

- Elias Gamma Coding is a universal, lossless compression method used to encode positive integers efficiently—especially useful in data structures like inverted indexes where many small numbers (like gaps between document IDs) occur.
- Elias Gamma Coding encodes a number ***x*** as:
- A unary representation of $\lfloor \log_2 x \rfloor + 1$
- Followed by the **remaining bits** of the binary representation of ***x***, excluding the leading 1.
- It strikes a balance between simplicity and efficiency and is particularly helpful in indexing and retrieval systems like web search engines.

- **Example: Take the number 13**

  Encoding:

  - **Step 1:** **Convert 13 to Binary**

    13 in binary is: **1101**

    This is a **4-bit** binary number.

  - **Step 2:** **Create the Prefix (Unary of the Length - 1)**

    The number of bits in 1101 is **4**, so the length minus 1 is **3**.

    Now, encode the number **4** (or length minus 1 = 3) in **unary coding**:

    Unary code of 4: 3 zeros followed by a 1 → **0001**

    This becomes the **prefix**.

  - **Step 3:** **Create the Suffix (Remove the Leading 1)**

    From the binary of 13 (1101), remove the leading 1: **101**

    This is the **suffix**.

  - **Step 4:** **Combine Prefix + Suffix**

    Now put the prefix and suffix together:

    Prefix: 0001
    Suffix: 101

    Gamma code: 0001101
    Elias Gamma Code for 13 is 0001101

Decoding:

1. Count leading zeros → **3 zeros**
2. Add a 1 → now you know the binary number has **4 bits**
3. Read the next 3 bits (101) and add the leading 1 → 1101 = **13**

C. Elias Delta Coding

- **Elias Delta Coding** is a universal, lossless integer compression method.
- It's more efficient than **Elias Gamma Coding** for large numbers and is often used when we need to encode **larger gap values** in inverted indexes.
- It works in **two layers**, essentially combining **Elias Gamma Coding** and standard binary representation.

- **Example: Elias Delta Coding for 13**
  Encoding:

  - **Step 1:** **Write *x* in binary**
    Let's denote the **length** of the binary representation of ***x*** as ***L***.
    13 in binary = **1101**
    So, the **length L = 4**

  - **Step 2:** **Encode L using Elias Gamma Coding**
    This will be the **prefix**.
    Elias Gamma Code for 4
    - 4 in binary = 100 → 3 bits
    - Unary of 3 = 001 (2 zeros + 1)
    - Suffix = remove leading 1 from 100 → 00

    So, Elias Gamma of 4 = **00100**
    This is the **prefix** of the Elias Delta code.

  - **Step 3:** **Remove the leading 1 from the binary of *x***
    Suffix of 13's Binary (Remove leading 1)
    13 in binary is 1101 → remove the leading 1 → 101
    This is the **suffix**

  - **Step 4:** **Concatenate the Gamma-coded L and the suffix**
    This is the final Elias Delta code.
    Prefix (Gamma of 4): 00100
    Suffix (from 1101): 101
    Final Elias Delta Code for 13 = 00100101

D. Golomb Coding

- **Golomb Coding** is a lossless compression method that encodes a positive integer based on a **divisor parameter *M***.
- It works best when values follow a geometric distribution (i.e., small values are more frequent).
- It splits a number into two parts:
  - **Quotient** (for division by ***M***) → encoded using **unary coding**
  - **Remainder** → encoded using **binary representation**

- **Example:**

  Encoding: Encode *x* = 10 using Golomb Coding with *M* = 3.

  - **Step 1:** **Compute the quotient and remainder**

    We subtract 1 from ***x*** because Golomb coding is usually defined for $x \geq 1$.

    x = 10, ***M*** = 3. Therefore, ***x*** − 1 = 9

Quotient $q = \lfloor (x-1)/M \rfloor$
Quotient $q = 9 \div 3 = 3$
Remainder $r = (x-1) \mod M$
Remainder $r = 9 \bmod 3 = 0$

- **Step 2:** **Encode the quotient *q* using unary coding**
  q zeros followed by a 1
  q = 3 → unary code = 0001

- **Step 3:** **Encode the remainder *r* in truncated binary**

  Choose the smallest number of bits to represent all possible remainders [0, M−1]

  Use a special rule if MMM is not a power of 2 (as below)

  Let:

  - $b = \lceil \log_2 M \rceil$
  - $L = 2^b - M$

  Then:

  - If $r < L$, encode r using $b-1$ bits
  - If $r \geq L$, encode $r+L$ using b bits

Truncated binary for remainder (M = 3)

$b = \lceil \log_2 3 \rceil = 2$

$L = 2^2 - 3 = 1$

And,

$r = 0 < L = 1$ → encode using $b - 1 = 1$ bit

So:

Remainder 0 → binary = 0

Therefore,

Final Golomb Code = 0001 (quotient) + 0 (remainder)
= 00010

E. Variable-Byte Coding

- Variable-Byte Coding is a technique that encodes integers using one or more bytes.
- Each byte has:
  - 7 bits for data (the actual number)
  - 1 control bit (the highest bit, called the *continuation bit*)
- The continuation bit tells the decoder whether the current byte is the last byte of the number or not.
  - If the highest bit is 1 → This is the last byte of the number.
  - If the highest bit is 0 → There are more bytes to read.

- **Example:**

  Encoding: Encode x = 130

  - **Step 1:** **Convert the number to binary.**
    Binary of 130 = **10000010**

  - **Step 2:** **Split the binary into 7-bit groups starting from the least significant bit (right side).**
    - **0000010 (7 bits)**
    - **0000001 (remaining bits)**

  - **Step 3:** **Add a continuation bit:**
    - For the last group, add a 1 at the front.
    - For all other groups, add a 0 at the front.

    So,
    - First group (0000001) → not last → 00000001
    - Second group (0000010) → last → 10000010

  - **Step 4:** **Write the bytes from left to right (most significant first).**
    We reverse the order (most significant byte first):
    00000001 10000010
    So, the encoded bytes for 130 are:
    0x01 0x82

3.1.2 Latent Semantic Indexing

- **Latent Semantic Indexing (LSI)** is an advanced information retrieval technique used to overcome the **limitations of keyword-based search**, especially the problems of:
  - **Synonymy**: different words with similar meanings (e.g., "car" and "vehicle")
  - **Polysemy**: same word with multiple meanings (e.g., "bank" as in river vs. money)

- LSI helps search engines **understand the meaning** behind words by discovering hidden relationships between terms and documents.
- Instead of relying on exact word matches, LSI captures the **underlying concepts** by analyzing **patterns** in how words co-occur across documents.
- It then projects both documents and queries into a **semantic (concept) space**, where meaning—not just literal terms—is the focus.
- The goal of LSI is to improve **recall and accuracy** by retrieving documents that are **semantically relevant**, not just those with exact keyword matches.
- Traditional keyword-based retrieval fails to capture **semantic meaning**; different words (e.g., "image", "photo") may describe the same concept but are treated as unrelated.
- **LSI as a solution:**
  - Uses **statistical associations** between terms to find **latent (hidden) semantic structures** in the data.
  - Applies a mathematical technique called **Singular Value Decomposition (SVD)** to reduce noise and reveal deeper relationships between words and documents.
  - Maps both documents and queries into a **"concept space"**, where similar meanings are grouped even if different words are used.
- **How LSI works:**
  - Starts with a **term-document matrix (A)** of size $m \times n$, where:
    - m = number of distinct terms
    - n = number of documents
    - Each cell Aij = term frequency (or other weight like TF-IDF) of term i in document j
  - SVD is then applied to this matrix to reduce its dimensions and uncover the **semantic relationships**.

3.1.2.1 Singular Value Decomposition (SVD)

- Singular Value Decomposition (SVD) is a fundamental concept in linear algebra that breaks down a matrix into three simpler matrices:
  - a unitary matrix,
  - a diagonal matrix,
  - another unitary matrix.
- This decomposition reveals many useful properties of the original matrix, including its rank, how it behaves under linear transformations, and provides a way to approximate it with a lower-rank version.
- **The Decomposition:**
  Given a real matrix A (like a term-document matrix), SVD says:

$$\mathbf{A = U \cdot \Sigma \cdot V^T}$$

Where:

- **A** = the original matrix (e.g., words × documents)
- **U** = left singular vectors → tells how **terms** relate to concepts
- **Σ** = diagonal matrix of **singular values** → tells the **importance** of each concept
- $\mathbf{V^T}$ = right singular vectors → tells how **documents** relate to concepts

- **Singular Values:**
  - The singular values in Σ represent the "strengths" of the different transformations that A applies to data.
  - They are often arranged in descending order.
- **Left and Right Singular Vectors:**
  - The columns of U are called the left singular vectors, and the columns of V are called the right singular vectors.
  - They are orthonormal and span the column space and row space of A, respectively.
- **Applications:**

  SVD has many applications in various fields, including:

  - **Dimensionality Reduction:**
    SVD is used in Principal Component Analysis (PCA) to reduce the dimensionality of data by keeping only the most important singular values and their corresponding vectors.
  - **Data Compression:**
    By approximating a matrix with its lower-rank SVD, we can reduce its size while preserving important information.
  - **Recommender Systems:**
    SVD is used to predict user preferences in collaborative filtering.
  - **Image Processing:**
    SVD can be used to compress and decompress images.
  - **Numerical Analysis:**
    SVD is used to solve linear systems of equations and least-squares problems.
- In essence, SVD provides a way to break down a matrix into its constituent parts, revealing its fundamental properties and allowing for various powerful applications.

- **Example:**

  Suppose we have a very small **term-document matrix**:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

This means:

- 3 terms (rows)
- 2 documents (columns)

Step 1: Apply SVD

Using SVD, we decompose A into:

$$U = \begin{bmatrix} -0.5 & 0.76 \\ -0.5 & 0.65 \\ -0.7 & -0.03 \end{bmatrix} \quad \Sigma = \begin{bmatrix} 2.29 & 0 \\ 0 & 0.3 \end{bmatrix} \quad V^T = \begin{bmatrix} -0.68 & -0.73 \\ -0.73 & 0.68 \end{bmatrix}$$

Step 2: Dimensionality Reduction (Keep Top-k Concepts)

Let's keep **only the top 1 singular value** (the most important concept):

- Truncate Σ to 1 × 1: $\Sigma_k = [2.29]$
- Truncate U and V^T to match

Now we have a **reduced matrix**:

$$\mathbf{A_k = U_k \cdot \Sigma_k \cdot V_k^T}$$

- $\mathbf{U_k}$ tells how each **term** relates to the main concept.
- $\mathbf{V_k^T}$ tells how each **document** relates to that concept.
- Even if two documents don't share exact terms, they may be **close in concept space**.

This gives us a low-rank approximation of the original matrix, capturing the **core semantic structure** with less noise.

- **Real-Life Example: Document Search**

Imagine you have this matrix:

Term	Doc1	Doc2
picture	1	0
photo	1	1
image	0	1

- A keyword search for "picture" won't return Doc2.
- But after applying SVD and reducing it to a **concept space**, we can see that "picture", "photo", and "image" are all related, and now Doc2 may be retrieved—**even though it didn't use the exact word.**

3.1.2.2 Query and Retrieval

- **Query and Retrieval in LSI (Latent Semantic Indexing)**

  1. **Query Representation**:
     - A user query ***q*** is treated like a new document and is represented as a **column vector** in the original term space (like a column in matrix **A**).
  2. **Transformation to Concept Space**:
     - Since documents are transformed into a **lower-dimensional k-concept space** using SVD, the query must be transformed too.
     - This is done using the formula:

$$q_k = q^T \cdot U_k \cdot \Sigma_k^{-1}$$

     - q_k is the **representation of the query in the concept space**.
  3. **Similarity Computation**:
     - For retrieval, we compare q_k with each document in the transformed space (rows of V_k).
     - A typical similarity measure is **cosine similarity**.
  4. **Alternative Method**:
     - Another approach is to compare the **scaled versions**:
       - Query:

$$\Sigma_k \cdot q_k^T$$

       - Documents:

$$\Sigma_k \cdot V_k^T$$

     - This includes the effect of the **singular values** (importance of concepts).
     - However, there's no clear evidence on whether this method outperforms the standard approach.

- LSI maps queries and documents into the same reduced concept space.
- Retrieval is based on similarity in that space, enabling discovery of semantically related documents—even if exact terms don't match.

Example:

- Consider the **term-document matrix** A with **3 terms** and **2 documents**:

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

  - Row 1: term1
  - Row 2: term2
  - Row 3: term3
  - Column 1: doc1
  - Column 2: doc2

- Now, we apply **SVD**:

$$A = U \cdot \Sigma \cdot V^T$$

- Let's assume (or simplify) the result of SVD gives:
  Matrices after SVD:

$$U = \begin{bmatrix} -0.71 & -0.71 \\ -0.71 & 0.71 \\ 0 & 0 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 1.73 & 0 \\ 0 & 1 \end{bmatrix}, \quad V^T = \begin{bmatrix} -0.71 & -0.71 \\ -0.71 & 0.71 \end{bmatrix}$$

- Now let's **reduce the dimensions** (choose top-1 concept → **rank-1 approximation**):

$$U_k = \begin{bmatrix} -0.71 \\ -0.71 \\ 0 \end{bmatrix}, \quad \Sigma_k = \begin{bmatrix} 1.73 \end{bmatrix}, \quad V_k^T = \begin{bmatrix} -0.71 & -0.71 \end{bmatrix}$$

- **Step-by-Step: Query Transformation and Retrieval**

Step 1: **Define the query**

- Let's say the **query** is similar to **doc1** → i.e., the query contains **term1 and term2**.
- So, the query vector ***q*** is:

$$q = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

Step 2: **Transform query into concept space**

Using:

$$q_k = q^T \cdot U_k \cdot \Sigma_k^{-1}$$

Step 2.1:

$$q^T \cdot U_k$$

$$q^T = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$$

$$U_k = \begin{bmatrix} -0.71 \\ -0.71 \\ 0 \end{bmatrix}$$

$$q^T \cdot U_k = (1)(-0.71) + (1)(-0.71) + (0)(0) = -1.42$$

Step 2.2: Multiply by

$$\Sigma_k^{-1}$$

$$\Sigma_k^{-1} = [1/1.73] \approx 0.578$$

$$q_k = -1.42 \cdot 0.578 \approx -0.82$$

Step 3: Compare with documents in concept space

From earlier:

$$V_k^T = \begin{bmatrix} -0.71 & -0.71 \end{bmatrix}$$

- So, doc1 and doc2 are both mapped to -0.71 in the concept space.

Now compute similarity (cosine similarity):

- Cosine similarity between q_k=−0.82 and each document =−0.71:

$$\text{cosine similarity} = \frac{(-0.82)(-0.71)}{\| -0.82\| \cdot \| -0.71\|} = \frac{0.5822}{0.82 \cdot 0.71} \approx 1$$

So, both documents are **equally similar** to the query in this example.

3.1.3 Web Search

- Due to the efficiency problem, latent semantic indexing is probably not used in Web search yet.
- Current search algorithms are still mainly based on the vector space model and term matching.
- A search engine works by **first crawling web pages**, then **parsing, indexing, and storing** the content.
- When a user submits a query, the engine uses the **index for fast and efficient retrieval**.

A. Parsing

- In a search engine, **parsing** involves analyzing the HTML of a web page to extract a stream of **tokens or terms** for indexing.
- This is typically done using tools like **YACC** and **Flex**.
- Additionally, some **pre-processing steps** (e.g., stemming, stop-word removal) may be applied **before or after parsing** to prepare the text for indexing.

B. Indexing

- In the **indexing** phase, a search engine creates an **inverted index** to support efficient retrieval.
- Multiple inverted indices may be built:

    - A **small index** is created using terms from **titles and anchor texts**, as these often describe pages well.
    - A **full index** includes **all text** on the page, including anchor text.
- Importantly, anchor text is indexed **for the page it links to**, not the one containing it.
- During search, the engine may first check the small index and only consult the full index if more results are needed.
- This improves **efficiency** and **relevance**.

C. Searching and Ranking

- When a user submits a query, the search engine follows three main steps:

    1. **Pre-process the query** (e.g., stopword removal, stemming).
    2. **Search the inverted index** to find pages containing all or most query terms.
    3. **Rank the results** and return the most relevant pages to the user.

- The **ranking algorithm** is a crucial part of the search engine, but commercial systems keep it private.

- Traditional Information Retrieval (IR) methods, like **cosine similarity**, rank documents based on content alone.
- However, for the **Web**, this isn't enough—most queries return **millions of relevant pages**, so content alone can't determine which are best.
- The key challenge is ranking: showing the **most useful and trustworthy** pages at the top.
- Unlike traditional IR, the Web lacks content quality control, as **anyone can publish anything**, regardless of accuracy or expertise.
- To address this, **hyperlinks** are used as a quality signal.
- A link from one page to another is seen as a **vote of confidence** or a sign of **authority**.
- Pages with more "votes" (or **in-links**) are likely higher in quality.
- This idea led to **link-based ranking algorithms**, like **PageRank**, which analyze the **link structure of the Web** to calculate a page's **reputation score**.
- Modern search engines combine both **content relevance** and **link-based quality** to rank pages effectively.

- **Content-based evaluation** assesses how relevant a web page is to a query by examining **where and how** the query terms appear.
- Key factors include:

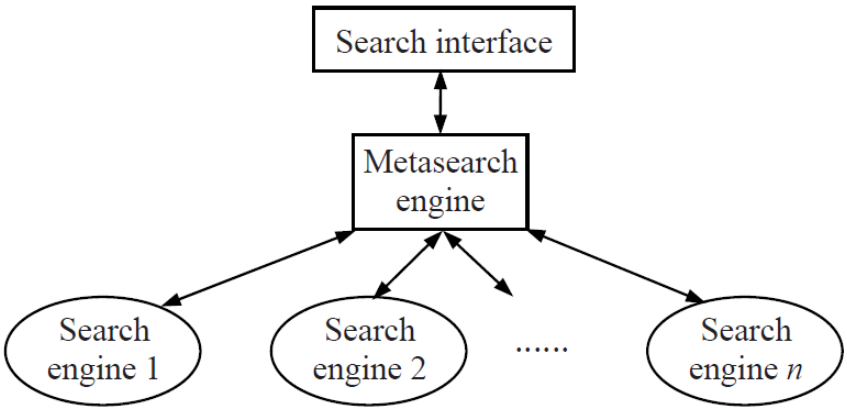
    1. **Occurrence Type** – Where the term appears:
        - **Title**: Appears in the page title.
        - **Anchor Text**: Found in links pointing to the page.
        - **URL**: Present in the web address.
        - **Body**: Found in the main content, with attention to **prominence** (e.g., bold, large font).

2. **Count** – How often the term appears in each section (e.g., 2 times in the title).
3. **Position** – The location of each term within a section, important for:
4. **Proximity**: Closer terms are more relevant.
5. **Sequence**: Terms appearing in the same order as the query are favoured.

- These factors help determine the page's **relevance score** based on both presence and contextual significance of query terms.
- To compute a page's **content-based score (IR score)**:
  1. **Each occurrence type** (title, body, URL, etc.) is assigned a **fixed weight**.
  2. **Raw term counts** are converted into **count-weights**.
  3. These are combined using a **dot product** of the **type-weight vector** and the **count-weight vector**.
- This gives the **IR score**, which is then **combined with a page's reputation score** (based on web link structure) to get the **final ranking score**.
- **For Different Types of Queries:**
  - **Single-word queries**: Simple IR score based on term counts and types.
  - **Multi-word queries**: More complex; considers **term proximity** (closer terms get higher weight).
    - **Type-proximity weights** are applied.
    - **Proximity values** are calculated.
    - The result is again a dot product that contributes to the IR score.
- **Term order** can also be included for further accuracy.
- The **final score** used for ranking combines both **IR relevance** and **page reputation**.

3.2 Meta Search

- A meta-search engine doesn't have its own database of web pages.
- Instead, it works by sending the user's query to multiple underlying search engines (called component search engines) and then combining (or merging) the results.
- **Working of Meta Search Engine:**
  1. User submits a query via the meta-search interface.
  2. The query is sent to several search engines.
  3. Results from each search engine are collected.
  4. These results are **fused** or **ranked together** into a single list.
  5. The combined results are presented to the user.
- This approach allows users to access a broader range of information by leveraging **multiple search engines** at once.



Meta-Search Architecture

- A meta-search engine offers several benefits, including increased search coverage by aggregating results from multiple search engines, each of which may cover only a small portion of the Web.
- This broader coverage ensures that relevant pages not found by a single search engine are included.
- Meta-search engines can also improve search effectiveness by reducing biases inherent in the ranking algorithms of individual search engines, leading to more precise results.
- Key operations in meta-search include identifying duplicate pages across different engines and combining the rankings.
- This involves comparing factors like domain names and titles to remove duplicates without downloading full pages.
- Additionally, meta-search algorithms fuse rankings using either similarity scores returned by the search engines or without them.
- These fusion techniques can also be applied within a single search engine to combine different similarity scores, ensuring more accurate results by considering various ranking factors.

3.2.1 Using Similarity Scores

- In a meta-search system, a set of candidate documents, $\boldsymbol{D = \{d_1, d_2, \ldots, d_N\}}$, is ranked using results from $\boldsymbol{k}$ underlying systems (component search engines or ranking techniques).
- Each system assigns a similarity score, $\boldsymbol{s_{ij}}$, to each document $\boldsymbol{d_j}$.
- Several methods are proposed for combining these scores:

1. **CombMIN**:
   The combined similarity score for each document dj is the minimum similarity score across all systems:

$$\text{CombMIN}(d_j) = \min(s_{1j}, s_{2j}, \ldots, s_{kj})$$

2. **CombMAX**:
   The combined similarity score is the maximum similarity score across all systems:

$$\text{CombMAX}(d_j) = \max(s_{1j}, s_{2j}, \ldots, s_{kj})$$

3. **CombSUM**:
   The combined similarity score is the sum of the similarity scores from all systems:

$$\text{CombSUM}(d_j) = \sum_{i=1}^{k} s_{ij}$$

4. **CombANZ**:
   The combined score is the sum of similarities divided by the number of non-zero similarities (systems that retrieved the document):

$$\text{CombANZ}(d_j) = \frac{\text{CombSUM}(d_j)}{r_j}$$

   where r_j is the number of non-zero similarities, or the number of systems that retrieved d_j.

5. **CombMNZ**:
   The combined score is the product of CombSUM and the number of non-zero similarities:

$$\text{CombMNZ}(d_j) = \text{CombSUM}(d_j) \times r_j$$

   where r_j is the number of non-zero similarities, or the number of systems that retrieved d_j.

- These methods aim to merge the results from multiple systems to produce a final ranking of documents.

3.2.2 Rank Positions

- This section discusses popular rank combination methods that rely solely on the rank positions from multiple search engines, inspired by **social choice theory**, which studies voting-based decision-making.
- Three key methods are described:

1. **Borda Ranking** (proposed by Jean-Charles de Borda in 1770):
   - Each voter (search engine) ranks the candidates (documents), assigning n points to the top-ranked, n–1 to the second, and so on (where n is the total number of candidates).
   - Unranked candidates share the remaining points evenly.
   - The candidate with the highest total score wins.

2. **Condorcet Ranking** (proposed by Marquis de Condorcet in 1785):
   - This method uses **pair-wise comparisons**.
   - A candidate wins if it beats all others in head-to-head matchups.
   - If unranked by a voter, the candidate is considered to lose to all ranked ones.
   - Unranked candidates are treated as tied.
3. **Reciprocal Ranking**:
   - Each candidate's score is the sum of the reciprocals of its rank across voters (e.g., 1 for first place, 1/2 for second, 1/3 for third, etc.).
   - Unranked candidates are ignored for that voter.
   - This method emphasizes top-ranked candidates more heavily than Borda ranking.

- These techniques provide ways to merge multiple ranked lists (like those from different search engines) into a single consensus ranking.

Example:

- This example illustrates how different rank combination methods work in a meta-search setting using rankings from five underlying search engines for four documents: **a, b, c,** and **d**. Each system ranks the documents differently:
  - **System 1**: a, b, c, d
  - **System 2**: b, a, d, c
  - **System 3**: c, b, a, d
  - **System 4**: c, b, d
  - **System 5**: c, b
- The purpose is to compute an overall score **Score(x)** for each document **x** (a, b, c, d) using various rank combination methods such as **Borda Count**, **Condorcet**, or **Reciprocal Rank**.
- This example demonstrates how aggregated rankings from multiple search engines can help determine the most relevant document by combining preferences and resolving inconsistencies in individual rankings.

Borda Ranking:

- In **Borda Ranking**, each document receives points based on its position in each search engine's ranking.
- Higher-ranked positions get more points.
- Using this method, the scores are:

  Score(*a*) = 4 + 3 + 2 + 1 + 1.5 = 11.5
  Score(*b*) = 3 + 4 + 3 + 3 + 3 = 16
  Score(*c*) = 2 + 1 + 4 + 4 + 4 = 15
  Score(*d*) = 1 + 2 + 1 + 2 + 1.5 = 7.5

- So, the final ranking is: **b > c > a > d**. This shows document **b** is considered most relevant overall across the systems.

Condorcet Ranking

- **Condorcet Ranking** is a method that ranks items based on pairwise comparisons.
- For each pair of candidate pages, we count how many times one is ranked above the other across different systems.
- These comparisons form an **n×n matrix** (where *n* is the number of candidates), recording wins, losses, and ties between every pair.

	a	*b*	*c*	*d*
a	-	1:4:0	2:3:0	3:1:1
b	4:1:0	-	2:3:0	5:0:0
c	3:2:0	3:2:0	-	4:1:0
d	1:3:1	0:5:0	1:4:0	-

The pair-wise comparison matrix for the four candidate pages

- From this matrix, a **win/lose/tie table** is created for each page.

	win	*lose*	*tie*
a	1	2	0
b	2	1	0
c	3	0	0
d	0	3	0

The win, lose and tie table for the comparison matrix

- Pages are ranked based on the number of pairwise wins.
- If there's a tie in wins, losses are used to break the tie, and further ties are broken arbitrarily.
- In the given example, page **c** wins all its pairwise comparisons, making it the **Condorcet winner**, followed by **b**, **a**, and **d**.
- **Final ranking: c > b > a > d.**

Reciprocal Ranking

- A candidate's score is calculated by summing the reciprocals of its rank positions across all voters (e.g., 1 for 1st place, 1/2 for 2nd, 1/3 for 3rd, and so on).
- Candidates that are not ranked by a voter are excluded from that voter's calculation.
- For the same example:

  Score(a) = 1 + 1/2 + 1/3 = 1.83
  Score(b) = 1/2 + 1 + 1/2 + 1/2 + 1/2 = 3
  Score(c) = 1/3 + 1/4 + 1 + 1 + 1 = 3.55
  Score(d) = 1/4 + 1/3 + 1/4 + 1/3= 1.17

- **Final ranking: c > b > a > d.**

3.3 Web Spamming

- Web search plays a crucial role in today's information age, as higher visibility on search engines can lead to financial success and recognition.
- The ranking of web pages is a key factor in this visibility—pages ranked low are often overlooked by users.
- Consequently, understanding and optimizing for search engine ranking algorithms has become essential for businesses and individuals.
- However, this has also led to the rise of web spamming, where pages are manipulated to appear more important to search algorithms without actually increasing their informational value.
- This manipulation exploits the fact that search engines rely on surface-level features, rather than truly understanding content.
- While some optimization efforts are legitimate, others cross the line into unethical spamming, negatively affecting users and search engines alike.
- Spam clutters search results, wastes resources, and degrades the quality of information found online.
- SEO companies have emerged to assist in improving rankings, with practices ranging from ethical optimization to manipulative tactics.

3.3.1 Content Spamming

- Most search engines use TF-IDF-based methods to determine how relevant a page is to a user's query.
- Content-based spamming, also known as term spamming, manipulates the text in various parts of an HTML page to falsely increase a page’s relevance.
- Since TF-IDF relies on term frequency, spammers insert targeted keywords into different fields of a webpage to boost rankings without improving actual content quality.
- Term spamming can occur in various parts of a webpage to manipulate search rankings.
- Spammers often target the **title**, as search engines give it significant weight.
- **Meta-tags**, which provide page metadata, are frequently abused, leading many search engines to ignore or devalue them.
- The **body** of a page is also used to insert spam terms.
- Additionally, **anchor text**—the clickable text in hyperlinks—is a key ranking factor and can be exploited to influence both the linking and linked pages.
- Lastly, some search engines analyze **URLs**, so spammers may insert keywords directly into the web address to boost visibility.

- There are two main techniques used in term spamming, both involving the creation of synthetic content containing spam terms:

  1. **Repeating important terms**:
     - This approach boosts the term frequency (TF) of specific keywords to increase a page's relevance.
     - Instead of plainly repeating terms—an easily detectable tactic—spammers insert the terms into sentences, often copied from other sources.
     - For example, the word “mining” might be placed randomly within a sentence like “the picture mining quality of this camera mining is amazing” to avoid detection.
  2. **Dumping many unrelated terms**:
     - This method aims to make a page appear relevant to a broad range of queries by including a large variety of terms.
     - Spammers typically copy and merge content from different web pages to quickly generate spammy content.
     - Additionally, some advertisers exploit frequently searched terms (e.g., “Tom Cruise”) by inserting them into unrelated pages, such as those promoting cruise vacations, to attract more user traffic.

3.3.2 Link Spamming

- Hyperlink spamming targets the reputation score of a page, which is influenced by link structure.

Out-link Spamming

- **Out-link spamming** involves adding numerous links from a spammer's page to authoritative pages to falsely boost its hub score.
- Spammers may use **directory cloning**, where they replicate sections of established web directories (like Yahoo! or DMOZ) to quickly generate a large number of outbound links, creating the appearance of a high-quality hub page.

In-link Spamming

- **In-link spamming** involves manipulating incoming links to a page to falsely increase its reputation score, which is more difficult than out-link spamming since it requires links from other websites. Spammers use several tactics:

  1. **Creating a honey pot**:
     - They build pages with genuinely useful content (e.g., glossaries or FAQs) to attract backlinks from real users.
     - These pages secretly link to spam pages, boosting their reputation.

2. **Submitting to web directories**:
    - Spammers submit spam page URLs to reputable directories, leveraging the authority of those sites to improve rankings.
3. **Posting on user-generated content platforms**:
    - They insert spam links into comments, forum posts, blogs, and reviews.
4. **Link exchange schemes**
    - Groups of spammers agree to link to each other's pages to mutually enhance rankings.
5. **Spam farms**
    - Spammers create and control many websites, constructing a custom link network to elevate target pages.

3.3.3 Hiding Techniques

- Spammers use various techniques to conceal spam content from web users while allowing search engines to index it.
- These techniques include:

1. **Content Hiding**:
    - Spam content is made invisible to users by using methods like matching the text color to the background or using hidden images.
    - For example:
        - *Text hiding*: spam items (white text on a white background).
        - *Hyperlink hiding*: (a blank image hides the hyperlink).
        - *Scripts*: Spammers may also use scripts to hide elements by altering HTML attributes, like setting visibility to false.
2. **Cloaking**:
    - A technique where spam servers show different content to users and search engine crawlers.
    - For example:
        - Servers identify crawlers by matching IP addresses or examining the user-agent field in HTTP requests (e.g., "User-Agent: Mozilla/4.0" for a browser).
        - This allows spam pages to be presented to search engines for indexing, while users see the intended content.
3. **Redirection**:
    - Spammers redirect users to a different URL than what is indexed by search engines.
    - This is often done using:

- *Meta refresh tag*: <meta http-equiv="refresh" content="0;url=http://newpage.com"> (redirects instantly to a new page).
- *Scripts*: Scripts can also be used to automatically redirect users to another URL.

- These methods hide spam from users but ensure that the spam page is still indexed by search engines.

3.3.4 Combating Spam

- Here are some of the key points on combating the spam:

1. **Redirection and Cloaking**:
   - Redirection using the "refresh" meta-tag is easy to detect, but redirection via scripts is harder for search engines to identify because crawlers do not execute scripts.
   - To prevent cloaking, search engine crawlers may identify themselves as regular web browsers.
2. **Anchor Text and PageRank**:
   - Using anchor text from other pages helps combat content spam because it is considered more trustworthy.
   - The PageRank algorithm can reduce content spam by considering the reputation of linking pages.
   - However, it does not address in-link spamming methods.
3. **TrustRank**:
   - A method called TrustRank is proposed to combat all types of spam.
   - It works by distinguishing reputable and spam pages through link analysis, leveraging the fact that reputable pages rarely link to spam, and spam pages often link to high-quality pages.
4. **Spam Classification**:
   - Spam detection can be treated as a classification problem.
   - Features used to detect spam include:
     - Number of words on the page.
     - Average word length.
     - Number of words in the title.
     - Fraction of visible content.
     - Other features like anchor text, compressibility, and n-gram likelihoods are also used.
   - A classifier trained with these features has been shown to accurately detect spam pages.

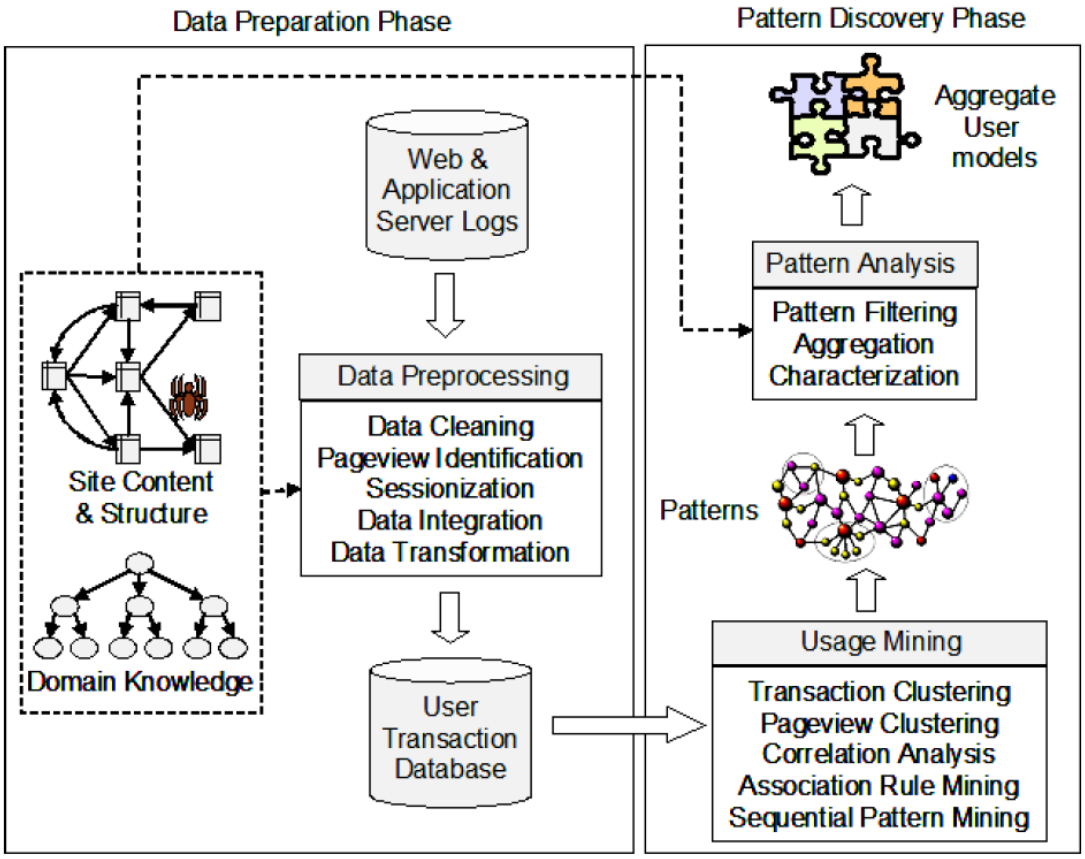
5. **Block-Based Ranking**:
    - Another technique is to partition web pages into blocks and assign different importance levels.
    - Links and terms in less important blocks are given lower weights in the PageRank computation.
    - This helps counteract link and term spamming, as spam links are often placed in less important areas of a page.
6. **Ongoing Battle**:
    - Despite these methods, sophisticated spam techniques are still hard to detect.
    - As search engines evolve, spammers continuously invent new methods to evade detection, making the fight against spam an ongoing challenge.

- Sophisticated spam remains difficult to detect, and combating it is an ongoing challenge.
- As search engines improve at detecting certain spam techniques, spammers continue to develop more advanced methods to bypass detection.

Chapter – 4: Web Usage Mining

4.1 Web Usage Mining

- As e-commerce and Web services grow, organizations are collecting massive amounts of clickstream, transaction, and user profile data.
- Analyzing this data helps businesses understand customer value, improve marketing, assess campaign effectiveness, personalize user experiences, and optimize website structure.
- This analysis involves automatically uncovering patterns and relationships from largely semi-structured data, typically found in server logs and operational databases.
- **Web usage mining** is the automated process of discovering and analyzing patterns in data such as clickstreams and user transactions, generated through interactions with websites.
- Its main goal is to model and understand user behavior by identifying commonly accessed pages or resources shared by users with similar interests or needs.

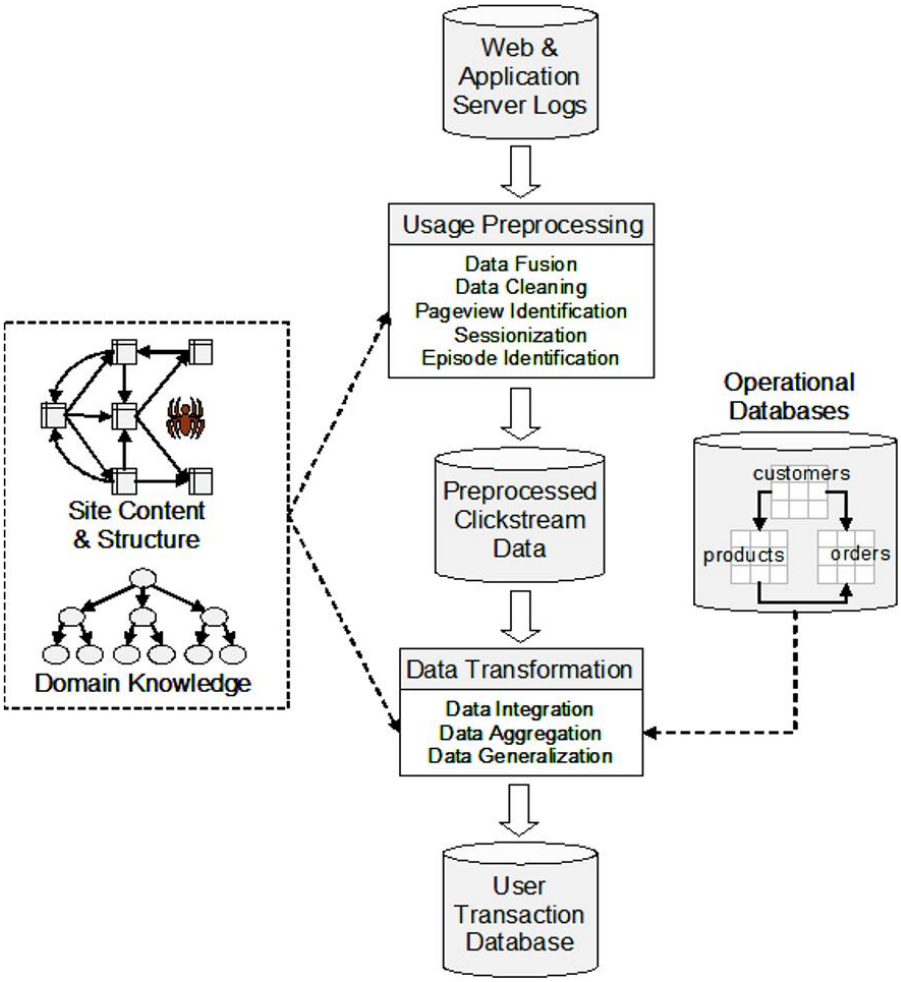


The Web Usage Mining Process

- The Web usage mining process consists of three main stages:
    1. **Data Collection and Pre-processing**:
        - Clickstream data is cleaned and organized into user sessions.
        - Additional information like site structure or domain knowledge may be incorporated to enrich the data.
    2. **Pattern Discovery**:
        - Techniques from statistics, databases, and machine learning are applied to uncover hidden patterns and generate summary statistics about user behavior and site usage.
    3. **Pattern Analysis**:
        - Discovered patterns are refined and used to build user models, which can support applications like recommendation systems, visualization tools, and Web analytics.
- These stages work together to extract meaningful insights from user interactions with websites.

4.2 Data Collection and Pre-Processing

- Creating a suitable target dataset is a crucial and often time-consuming part of Web usage mining.
- Due to the complexity and diversity of clickstream data—often gathered from multiple sources—the **data preparation** stage is critical.
- It involves:
    - Pre-processing raw data
    - Integrating data from various channels
    - Transforming data into formats suitable for mining
- This process requires specialized **algorithms and heuristics**, especially for tasks like data cleaning, user/session identification, and pageview recognition.
- Proper data preparation directly influences the success of mining useful patterns, particularly in e-commerce, where it also supports the extraction of valuable user and site metrics.



Steps in Data Preparation for Web Usage Mining.

4.3 Sources and Types of Data

- The main data sources for Web usage mining are server log files, including both Web server access logs and application server logs.
- Other essential sources include site files, meta-data, operational databases, application templates, and domain knowledge.
- Additional data can also come from client-side or proxy-level tracking, and external sources like ComScore, NetRatings, and Acxiom, which provide clickstream and demographic data.
- The data collected for Web usage mining can be grouped into four main categories.

A. Usage Data

- **Usage Data** is the primary data source in Web usage mining, automatically collected by web and application servers.
- Each user interaction with a website (an HTTP request) generates a log entry.

- These entries may include details like:
  - Time and date of the request
  - Client's IP address
  - Requested resource and parameters
  - Request status and HTTP method
  - User agent (browser and OS info)
  - Referring page
  - Cookies (to track returning visitors)
- This data provides a detailed view of user behavior and is essential for analyzing how users interact with web resources.
- Consider the example:

1	2006-02-01 00:08:43 1.2.3.4 - GET /classes/cs589/papers.html - 200 9221 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1;+SV1;+.NET+CLR+2.0.50727) http://dataminingresources.blogspot.com/
2	2006-02-01 00:08:46 1.2.3.4 - GET /classes/cs589/papers/cms-tai.pdf - 200 4096 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1;+SV1;+.NET+CLR+2.0.50727) http://maya.cs.depaul.edu/~classes/cs589/papers.html
3	2006-02-01 08:01:28 2.3.4.5 - GET /classes/ds575/papers/hyperlink.pdf - 200 318814 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1) http://www.google.com/search?hl=en&lr=&q=hyperlink+analysis+for+the+web+survey
4	2006-02-02 19:34:45 3.4.5.6 - GET /classes/cs480/announce.html - 200 3794 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1;+SV1) http://maya.cs.depaul.edu/~classes/cs480/
5	2006-02-02 19:34:45 3.4.5.6 - GET /classes/cs480/styles2.css - 200 1636 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1;+SV1) http://maya.cs.depaul.edu/~classes/cs480/announce.html
6	2006-02-02 19:34:45 3.4.5.6 - GET /classes/cs480/header.gif - 200 6027 HTTP/1.1 maya.cs.depaul.edu Mozilla/4.0+(compatible;+MSIE+6.0;+Windows+NT+5.1;+SV1) http://maya.cs.depaul.edu/~classes/cs480/announce.html

Example: Portion of a Typical Server Log

- This example illustrates how **web server logs** capture detailed user interactions:
  1. **Entry 1**:
     - A user with IP 1.2.3.4 accesses the page /classes/cs589/papers.html from an external referrer (dataminingresources.blogspot.com).
     - The browser and OS info are recorded in the agent field.

2. **Entry 2**:
   - The same user navigates to /classes/cs589/papers/cms-tai.pdf, with the referrer now being the previously visited papers.html.
3. **Entry 3**:
   - A different user reaches /classes/ds575/papers/hyperlink.pdf via a **Google search** using the query *"hyperlink analysis for the web survey"*.
4. **Entries 4–6**:
   - These all stem from a **single user click** on /classes/cs480/announce.html.
   - Entries 5 and 6 log automatic requests for **embedded images** in that page.

- This showcases how multiple log entries can correspond to a single user session or action, and how logs help track navigation paths and content access.

- In Web usage mining, data is transformed and aggregated based on the analysis goals, using different levels of abstraction:
  - **Pageview**:
    - The most basic unit, representing a single user action (e.g., clicking a link), and typically includes multiple web objects that together form what the user sees.
    - Each pageview reflects a specific *user event* like reading an article or adding a product to a cart.
  - **Session**:
    - A sequence of pageviews by a single user during one visit to the site.
    - It's the fundamental unit for analyzing user behavior over a visit.
  - **Session Abstraction**:
    - Sessions can be further refined by focusing only on significant or relevant pageviews for more targeted analysis.B.

B. Content Data

- **Content Data** in a website refers to the collection of objects and their relationships that are presented to users.
- It typically includes:
  - **Text and images** from sources like static HTML/XML pages, multimedia files, and dynamically generated content.
  - **Meta-data** such as keywords, document attributes, semantic tags, and HTTP variables.
  - **Domain ontology**, which provides structured knowledge like product hierarchies, semantic relationships using languages like RDF, or database schemas linked to the site's operational data.
- This data supports both the content shown to users and deeper semantic analysis.

C. Structure Data

- **Structure Data** refers to the way a website's content is organized from the designer's perspective.
- It includes:
  - **Inter-page structure**: The network of hyperlinks connecting different pages, often represented in a site map.
  - **Intra-page structure**: The layout within individual pages, such as the tag hierarchy in HTML/XML documents.
  - For **dynamically generated pages**, structure data can be harder to capture and may require tools that understand the scripts generating the content or simulate user interactions with parameters.
- Overall, structure data maps both the layout of pages and their relationships, aiding in understanding site navigation and design.

D. User Data

- **User Data** in web usage mining refers to additional information about users stored in the site's operational databases.
- This may include:
  - **Demographic data** (e.g., age, location) of registered users
  - **User activity history** like past purchases, visits, or item ratings
  - **Interests** gathered explicitly (e.g., user preferences) or implicitly (e.g., behavior tracking)
  - **Anonymous identifiers**, such as data from client-side cookies, which help recognize repeat visitors without knowing their identity
- This data is crucial for **personalization**, enabling websites to tailor content and experiences to individual users.

4.4 Data Modelling

- In Web Usage Mining (WUM), usage data pre-processing organizes raw web logs into a structured format to enable effective analysis into two essential sets:
  - **Pageviews (P):**
    - A set of distinct, semantically meaningful content items (like web pages, products, or articles).

$$P = \{p_1, p_2, \ldots, p_n\}$$

  - **User Transactions (T):**
    - A set of user sessions (visits), where each session is represented as a **subset** of pageviews from P.

$$T = \{t_1, t_2, \ldots, t_m\}$$

- Each transaction (user session) ***t*** is modeled as a **sequence** of pageviews along with weights showing their importance or relevance.
- Each transaction is written as a sequence of **ordered pairs**:

$$t = \{(pt_1, w(pt_1)), (pt_2, w(pt_2)), \ldots, (pt_l, w(pt_l))\}$$

  - pt_i A specific pageview visited during the session.
  - $w(pt_i)$ A weight that indicates the **significance** or **impact** of the pageview.

- This structure is important for later mining tasks like pattern recognition, recommendations, or trend analysis.

Determination of Weights:

- The weight $w(pt_i)$ tells how important or influential that pageview was in the session.
- These weights can be determined in various ways, depending on the mining task**.**

  1. Collaborative Filtering Use Case:
  - Weights are based on **explicit user ratings** (like 4 stars for a movie or product).
  - Helps in building **recommendation systems** by matching similar users.

  2. Binary Weights:
  - Weight = 1 if the user visited the page, 0 otherwise.
  - Common in simple presence/absence models.

  3. Time-Based Weights (Duration):
  - The time a user spent on each page.
  - The longer the time spent, the more engaged the user likely was.
  - Helps gauge **interest** or **engagement** levels.

- Because the time for the last pageview in a session is usually unknown, it is often estimated using the average time spent on that page in other sessions.
- To improve accuracy and reduce bias, weights are often normalized or log-transformed.
- This structured format supports effective pattern discovery, user profiling, and recommendation systems.

- For certain data mining tasks like **clustering** and **association rule mining**, where the order of pageviews in a session doesn't matter, each user transaction can be represented as a **vector** in an *n*-dimensional space—one dimension for each unique pageview.
- If a transaction ***t*** includes specific pageviews, its vector representation $\mathbf{t}$ is:

$$\mathbf{t} = (w^t_{p_1}, w^t_{p_2}, \ldots, w^t_{p_n})$$

- Here, $w_{p_i}^t$ is the **weight** (e.g., duration or binary value) associated with pageview p_i in transaction t.
- If a pageview p_i is not present in the transaction, its weight is set to zero.
- This approach allows all user sessions to be structured into an **m × n matrix**, called the **User-Pageview Matrix (UPM)**, where ***m*** is the number of user transactions and ***n*** is the number of unique pageviews.
- This matrix format is ideal for applying traditional data mining and machine learning techniques.

- Consider the example in the figure below:

Sessions / users	Pageviews: A	B	C	D	E	F
user0	15	5	0	0	0	185
user1	0	0	32	4	0	0
user2	12	0	0	56	236	0
user3	9	47	0	0	0	134
user4	0	0	23	15	0	0
user5	17	0	0	157	69	0
user6	24	89	0	0	0	354
user7	0	0	78	27	0	0
user8	7	0	45	20	127	0
user9	0	38	57	0	0	15

An example of a user-pageview matrix (or transaction matrix)

- A **user-pageview matrix** example (refer the figure) shows how much time each user spent on specific pages, with weights representing durations in seconds.
- In real applications, these weights are typically **normalized** to handle differences in user behavior and viewing habits.
- Sometimes, a pageview might represent a group of pages, and in such cases, the weights can be **composite values** reflecting that aggregation.
- Once this matrix of transactions is built, various **unsupervised learning techniques** can be applied.
- For instance:
  - **Clustering transactions** (sessions) helps identify distinct **user segments** based on similar browsing patterns.
  - **Clustering items** (pageviews) or using **association/sequential pattern mining** reveals meaningful **relationships** between pages—like which pages are often visited together or in a particular sequence.
- These analyses are essential for understanding user behavior, personalizing experiences, and improving website design or marketing strategies.

- **Semantic content** from web pages can be integrated into the **Web Usage Mining (WUM)** process to improve user analysis.
- Each pageview p is represented as an **I-dimensional vector** of textual or conceptual features extracted from the website, where ***r*** is the number of features in a global dictionary.
- The vector is defined as:

$$p = (fwp(f_1), fwp(f_2), ..., fwp(f_r))$$

  where $fwp(f_j)$ is the weight of feature f_j in pageview ***p***.

- This results in an **n × r pageview-feature matrix (PFM)** for all pageviews in the site.
- By transforming traditional user sessions from being simple sequences of pageviews into **"content-enhanced" transactions** (i.e., vectors of semantic features), the mining process captures not just where a user went, but also **what concepts or topics** they engaged with.
- This approach enables a deeper understanding of user interests and improves applications like personalization, recommendation, and user profiling.
- The most straightforward way to transform user transactions is by mapping each pageview to one or more content features, which can represent concepts or categories.
- This transformation is mathematically represented by multiplying the **user-pageview matrix (UPM)** with the **pageview-feature matrix (PFM)**.
- The result is a new matrix (TFM), where each row is a feature vector that represents a user's transaction in terms of content features—essentially capturing their interests in specific topics.

- An example of content-enhanced transactions is illustrated, in the figure below, using two matrices: a **user-pageview matrix**, which shows which pages users visited, and a **term-pageview matrix**, which represents the concepts or terms found on each page.
- The **transpose** of the term-pageview matrix forms the **pageview-feature matrix** used in the transformation process.

	A.html	B.html	C.html	D.html	E.html
user1	1	0	1	0	1
user2	1	1	0	0	1
user3	0	1	1	1	0
user4	1	0	1	1	1
user5	1	1	0	0	1
user6	1	0	1	1	1

	A.html	B.html	C.html	D.html	E.html
web	0	0	1	1	1
data	0	1	1	1	0
mining	0	1	1	1	0
business	1	1	0	0	0
intelligence	1	1	0	0	1
marketing	1	1	0	0	1
ecommerce	0	1	1	0	0
search	1	0	1	0	0
information	1	0	1	1	1
retrieval	1	0	1	1	1

Examples of a user-pageview matrix (top) and a term-pageview matrix (bottom)

- In this example, binary weights are used for simplicity, but in real scenarios, weights typically reflect the importance of a page in a session or the frequency of terms in a page.
- The content-enhanced transaction matrix is created by multiplying the **user-pageview matrix** with the **transpose of the term-pageview matrix**.
- This resulting matrix (shown in figure below) represents user transactions in terms of content features or concepts, highlighting users' interests based on the content of the pages they visited.

	web	data	mining	business	intelligence	marketing	ecommerce	search	information	retrieval
user1	2	1	1	1	2	2	1	2	3	3
user2	1	1	1	2	3	3	1	1	2	2
user3	2	3	3	1	1	1	2	1	2	2
user4	3	2	2	1	2	2	1	2	4	4
user5	1	1	1	2	3	3	1	1	2	2
user6	3	2	2	1	2	2	1	2	4	4

The content-enhanced transaction matrix from matrices of previous figure.

- When content features include relational or semantic attributes (like actors, genres, or directors), web usage mining can uncover deeper insights into user interests.
- For example, analyzing a movie website could reveal patterns such as users interested in "British," "Romance," and "Comedy" movies also liking "Hugh Grant."
- This shows how combining semantic content with usage data can uncover meaningful relationships between user preferences and item properties.

4.5 Discovery and Analysis of Web Usage Patterns

- The type and depth of analysis applied to integrated web usage data vary based on the analyst's goals and desired outcomes.
- This section outlines the most common pattern discovery and analysis techniques used in web usage mining, along with their practical applications.

4.5.1 Session and Visitor Analysis

- Statistical analysis of pre-processed session data is the most common approach in web usage analysis.
- It involves aggregating data by units like days, sessions, or visitors and applying standard statistical methods to understand user behavior.
- This method, used by most commercial web log analysis tools, provides insights such as popular pages, average view time, common navigation paths, and entry/exit points.
- Although less in-depth, it offers valuable information for system optimization and marketing.
- Additionally, modern tools are beginning to integrate data mining techniques for more advanced insights.
- Online Analytical Processing (OLAP) is another method for analyzing integrated web usage data, offering greater flexibility and depth.
- It uses a **multidimensional data warehouse** that combines usage, content, and e-commerce data at various aggregation levels.
- OLAP tools allow users to adjust the level of detail across dimensions like time, domain, resource, user agent, and referrer.
- This enables focused or high-level analysis, such as examining traffic during specific periods or exploring URL structures. Integrating e-commerce data enhances business intelligence, and OLAP results can also feed into data mining or visualization tools for deeper insights.

4.5.2 Cluster Analysis and Visitor Segmentation

- Clustering is a data mining technique used in web usage mining to group similar items—in this case, user sessions or web pages.
- There are two main types:
  - **User clustering**: Groups users with similar browsing behavior, helping with market segmentation, demographic analysis, and personalized content delivery.
  - **Page clustering**: Groups pages with similar content or usage patterns.
- User sessions can be represented as vectors in a multidimensional space (based on pageviews), and clustering algorithms like **k-means** group these into segments.
- While these clusters reflect user behavior patterns, individual clusters may be large and complex.

- To simplify analysis, each cluster can be summarized using a **centroid**—a mean vector representing average behavior in that group.
- This approach supports business intelligence, personalization, and creating user communities, offering insights for targeted recommendations and improved user experience.
- User transactions can be represented as vectors in a multi-dimensional space based on pageviews.
- Clustering algorithms like **k-means** group these transactions into clusters based on similarity.
- These **transaction clusters** help identify segments of users with similar browsing behavior.
- However, because each cluster may contain thousands of transactions and pageviews, they don't easily reveal overall patterns.
- The main goal of clustering is to enable **analysis of user segments** for applications like **business intelligence** and **personalization**.

- A straightforward method to summarize a cluster is to calculate its **centroid** (mean vector), where each pageview's value is the ratio of its total weight across transactions to the number of transactions in the cluster.
- If pageview weights are binary, the centroid value indicates the percentage of transactions in which a pageview occurred, reflecting its significance in the cluster.
- By sorting these values, lower-weight pageviews can be filtered out, creating an **aggregate usage profile** that represents the interests or behavior of a significant user group.

- Formally, for a given transaction cluster cl, the **aggregate profile** pr_{cl} can be defined as a set of pageview-weight pairs derived from the **centroid** of the cluster:

$$pr_{cl} = \{(p, weight(p, pr_{cl})) \mid weight(p, pr_{cl}) \geq \mu\}$$

Where:

- $weight(p, pr_{cl})$ represents the importance of page p within the aggregate profile and is calculated as:

$$weight(p, pr_{cl}) = \frac{1}{|cl|} \sum_{s \in cl} w(p, s)$$

  - $|cl|$ is the total number of transactions in the cluster.
  - $w(p,s)$ is the weight of page p in a specific transaction s within the cluster.
  - The **threshold** μ is used to include only those pages that appear frequently enough in the cluster, filtering out less significant ones.

- This process results in a concise profile that captures the most representative pages for the user group within the cluster.
- Each aggregate profile can be represented as a vector in the original pageview space.
- This representation is useful for predictive modeling and recommender systems.

- For a new user, ***u***, who has visited a set of pages P_u, the system can compare P_u to existing profiles, identify similar ones, and recommend pages from those profiles that the user hasn't visited yet.

- The example in figure below illustrates how binary transaction data can be clustered (e.g., using k-means) to form user segments.

		A	B	C	D	E	F
Cluster 0	user 1	0	0	1	1	0	0
	user 4	0	0	1	1	0	0
	user 7	0	0	1	1	0	0
Cluster 1	user 0	1	1	0	0	0	1
	user 3	1	1	0	0	0	1
	user 6	1	1	0	0	0	1
	user 9	0	1	1	0	0	1
Cluster 2	user 2	1	0	0	1	1	0
	user 5	1	0	0	1	1	0
	user 8	1	0	1	1	1	0

Aggregated Profile for Cluster 1	
Weight	Pageview
1.00	B
1.00	F
0.75	A
0.25	C

Derivation of aggregate profiles from Web transaction clusters

- In **Cluster 1**, **pageviews B and F** are the most prominent, indicating strong user interest, while **pageview C**, appearing only once, may be excluded based on a filtering threshold.
- These patterns help define user segments—this one show high interest in **B and F**, and moderate interest in **A**.
- If a new user shows interest in **A and B**, the system might infer they belong to this segment and recommend **F**.
- Additionally, more advanced clustering methods can improve on k-means by handling noise, reducing sensitivity to initialization, and automatically determining the number of clusters.

- Page (or item) clustering can be done using either **usage data** (from user sessions or transactions) or **content features** (like keywords or product attributes).
  - **Content-based clustering** groups items by topic or category.
  - **Usage-based clustering** groups items that are often accessed or purchased together, helping create dynamic or static recommendations (e.g., related links or products).
- Recent advances use **stochastic methods**—like **mixture models**—to capture more complex, dynamic user behavior.
- These models are better suited for large, evolving websites where user behavior varies significantly and can't be easily represented by simple probability distributions.

- They help model different user "types" or tasks, each reflected in its own behavioral pattern.

- **Mixture models**, such as mixtures of Markov models, assume that user behavior can be grouped into ***k*** distinct types or clusters.
- Each user session is generated by first selecting a user cluster (with some probability), then modeling the session using a Markov model specific to that cluster.
- The **EM algorithm** is typically used to estimate both the cluster probabilities and the parameters of each model.
- These models offer flexibility by both clustering user sessions and describing navigation behavior within each cluster—for instance, identifying common browsing paths.
- Variants like **mixtures of hidden Markov models** can also model complex clickstreams and support tasks like automatic page classification.
- However, a key limitation is that each session is assumed to come from just one component model.
- This can reduce the model's ability to fully capture complex behaviors and may lead to **overfitting**.

- **Probabilistic Latent Semantic Analysis (PLSA)** addresses the limitations of traditional mixture models in modeling web user navigation.
- Unlike mixture models, where each session is tied to a single behavior type, PLSA assumes that each **user-page interaction** is generated through multiple **hidden variables** that explain user behavior.
- The process works as follows:
    1. A user is selected.
    2. A hidden variable (representing a latent behavior or interest) is chosen based on the user.
    3. A page is then selected based on that hidden variable.
- This allows each user to be associated with **multiple hidden behaviors**, reducing the risk of overfitting.
- PLSA also uses the **EM algorithm** to estimate parameters and uncover relationships between users, pages, and the underlying hidden factors driving their interactions.
- This method—based on **Probabilistic Latent Semantic Analysis (PLSA)**—offers a flexible framework for modeling relationships between users, items, and hidden (latent) factors that explain user behavior.
- Each user-item interaction is influenced by multiple latent variables, allowing for richer representation of behavior patterns.
- The model estimates the probability of a user interacting with an item by combining probabilities over these latent factors.

- It uses weights to represent the strength of each interaction (like page importance or item ratings), and the parameters are learned by maximizing a likelihood function over the observed data.
- This approach captures complex relationships and helps uncover meaningful patterns in user behavior.

- The **Expectation-Maximization (EM) algorithm** is used in the PLSA model to estimate parameters by maximizing likelihood. The algorithm alternates between two steps:
  - **Expectation step**:
    Computes the posterior probabilities of latent variables based on current estimates.
  - **Maximization step**:
    Re-estimates the parameters.
- This iterative process increases the likelihood of observed data until a local optimal solution is found. One major advantage of PLSA in web usage mining is its ability to use probabilistic inference to uncover relationships between users, pages, and interactions. This flexibility allows the model to capture various types of usage patterns.

4.5.3 Association and Correlation Analysis

- Association rule discovery and statistical correlation analysis help identify groups of items or pages commonly accessed or purchased together.
- This enables websites to optimize content organization and offer effective cross-sell recommendations.
- The **Apriori algorithm** is commonly used for association discovery, finding frequent itemsets in user transactions based on a minimum support threshold.
- From these itemsets, association rules are generated if they meet a specified minimum confidence threshold.
- Association rules are expressions of the form X → Y [sup, conf], where X and Y are itemsets, "sup" is the support (the probability that X and Y appear together), and "conf" is the confidence (the conditional probability that Y appears given X).
- Association rule mining in Web transaction data offers several advantages.
- For instance, high-confidence rules like
  "special-offers/ → shopping-cart/"
  can suggest successful marketing campaigns, and rules like "A → B" can help optimize site structure by indicating useful direct links between pages.
- These techniques are widely used in Web personalization and recommender systems, especially for making product or page recommendations.

4.5.4 Analysis of Sequential and Navigational Patterns

- Sequential pattern mining identifies patterns in Web usage data where a set of items (e.g., web pages) is followed by another item in a time-ordered sequence of sessions.
- This technique helps Web marketers predict future visit patterns, enabling more effective advertisement placement.
- It can also be used for trend analysis, detecting changes, and identifying similarities.
- In Web usage, sequential pattern mining captures frequent navigational paths followed by users.
- Sequential patterns (SPs) in Web usage data identify common sequences of web pages visited by users in a specific order.
- These patterns are defined by items that frequently occur together in a large number of transactions.
- A sequence occurs in a transaction if the items appear in the same order, and contiguous sequential patterns (CSPs) require that adjacent items appear consecutively within a transaction.
- In contrast, normal sequential patterns can represent non-contiguous item sequences that appear in the data.
- Given a sequence transaction set T, the support (denoted by $sup(S)$) of a sequential (respectively, contiguous sequential) pattern S in T is the fraction of transactions in T that contain S.
- The confidence of the rule $X \rightarrow Y$, where X and Y are (contiguous) sequential patterns, is defined as:

$$conf(X \rightarrow Y) = sup(X \circ Y) / sup(X)$$

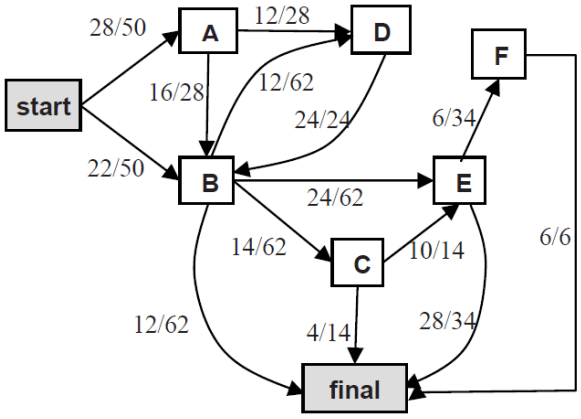
where ° denotes the concatenation operator.

- In Web usage data, Contiguous Sequential Patterns (CSPs) capture frequent navigational paths where items are adjacent, while Sequential Patterns (SPs) represent more general navigational patterns that don't require adjacency.
- These patterns help model user navigation, often using Markov models, where each pageview or category is a state, and the transition probabilities between states show how likely users are to navigate from one page to another.
- Markov models are useful for predictive tasks, such as predicting a user's next action based on previous behavior, link prediction, and Web prefetching.
- These models can also identify high-probability user navigational trails on a site.
- In a Markov model, the transition probabilities between states are used to predict the likelihood of future events, with the order of the model defining how many previous events influence the prediction.

- Web transactions can be modeled using Markov models, where pageviews (A, B, C, D, E) represent states, and the transitions between them capture the likelihood of one pageview following another.

- For example, the transition probability from the state A to B is 16/28 = 0.57 since out of the 28 occurrences of A in transactions, in 16 cases, B occurs immediately after A.

Transaction	Frequency
A, B, E	10
B, D, B, C	4
B, C, E	10
A, B, E, F	6
A, D, B	12
B, D, B, E	8

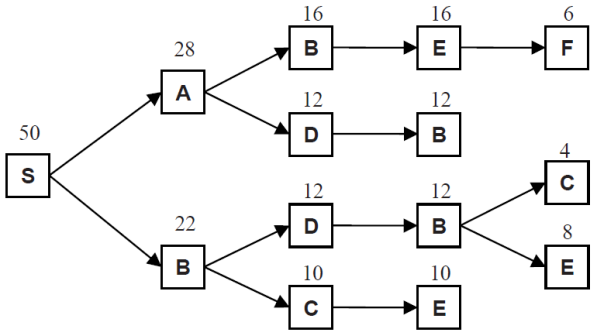


An example of modeling navigational trails as a Markov chain

- Higher-order Markov models, which consider more prior events to predict the next, can improve prediction accuracy but at the cost of lower coverage and increased complexity.
- To address these issues, approaches like all-kth-order Markov models and longest repeating subsequences (LRS) have been proposed to reduce model size and improve coverage.
- Additionally, selective Markov models use pruning techniques, such as support and confidence thresholds, to reduce complexity by eliminating less relevant states.

- A trie structure, like the **aggregate tree** used in the **WUM (Web Utilization Miner)** system, offers an efficient way to represent **contiguous navigational trails** from Web logs.
- In this approach, user sessions are transformed into sequences and inserted into a trie, where common prefixes are merged.
- Each node in the tree reflects a page in a sequence and is labeled with how often that navigational path appears in the data.
- WUM also includes **MINT**, a query language that allows the discovery of **generalized navigational patterns** using customizable constraints (e.g., wildcards, support, and confidence thresholds).
- This method is particularly useful for analyzing and improving the **navigational structure** of websites.

- This example explains how **WUM's aggregate tree** (a trie structure) is used to efficiently discover **navigational patterns** in Web usage data.
- Each node in the tree represents a navigational sequence and is labeled with its **frequency** in session data.



An example of modeling navigational trails in an aggregate tree

- For instance, the support and confidence of a sequence like <A, B, E, F> can be easily calculated from the node annotations:
  - **Support** = frequency of last item / total sessions (e.g., 6/50 = 0.12)
  - **Confidence** = frequency of last item / frequency of predecessor (e.g., 6/16 = 0.375)
- If the same sequence appears in multiple branches, their frequencies are **summed** to compute total support and confidence.
- For example, <D, B> has support (12+12)/50 = 0.48 and confidence 24/24 = 1.0.
- **Advantage**: Efficient pattern search with direct access to support and confidence.
- **Disadvantage**: Potentially **high space complexity**, especially for websites with many dynamic pages.

4.5.5 Classification and Prediction based on Web User Transactions

- **Classification** involves assigning data items to predefined categories.
- In the **Web domain**, it's used to build **user profiles** for specific classes or groups.
- It requires **feature extraction and selection** to represent class characteristics.
- Common **supervised learning algorithms** include:
  - Decision trees
  - Naive Bayes
  - k-nearest neighbor (k-NN)
  - Support Vector Machines (SVMs)
- Additionally, **previously discovered clusters or association rules** can aid in classifying new users.
- In essence, classification helps identify and categorize user behavior for personalization or targeted content.
- Classification techniques are crucial in **Web analytics** for modeling users based on predefined criteria.
- For example, users can be classified by their **likelihood to make purchases** using features like **demographics** and **browsing behavior**.

- Solving classification problems typically involves:
  1. Defining it as a classification task.
  2. Selecting or creating relevant features.
  3. Applying **supervised learning** or **regression** methods.
- A key area of interest within classification is **recommender systems**, which have broad applications and are a major focus of research.
